P04: Makers Makin' It, Act II -- The Sequel

PUSHEEN: EVAN KHOSH, MEGAN KWOK, ARTEMIS LEE, JOYCE LIN, SOPHIA CHI

DESIGN DOCUMENT

TARGET SHIP DATE: 2026-04-22

APP OVERVIEW:

A web application that allows users to visualize Instagram post data. Users will be able to compare how different variables affect different metrics, like the content category of a post and the amount of likes it gets. We are telling the story of how Instagram rewards/punishes certain post-archetypes and the type of media that gets more attention on the platform.

ROLES:

Possible responsibilities: Database, Python, FEF, HTML

1. MEGAN KWOK (PM, Python, HTML)
2. EVAN KHOSH (Database, Python)
3. ARTEMIS LEE
4. SOPHIA CHI (HTML, Python)
5. JOYCE LIN (HTML, FEF)

PROGRAM COMPONENTS:

Language: Python

- *__init__.py*
  - Utilizes Flask to serve our app on our nginx servers as well as facilitates communication between our frontend and backend components (including other existing python files as well as javascript files).
- *data.py*
  - Will contain calculations necessary for processing our datasets in accordance with user inputs. The outputted data points will be fed back to __init__.py and subsequently displayed in our graphs.

        - Will contain all MongoDB communications, pulls out data in response to calls from functions involved in math.py), storing of new data (gained from running our simulations) and all necessary manipulations
    - mongoSetup.py
        - Sets up MongoDB collections and pulls all of the data we will be using from Kaggle

Framework: Flask

Our go-to framework, and the one we're most familiar with, served on Nginx. Will support the communication of information between pages as well as allow data to be fed into our chosen data visualization library.

DB: MongoDB

Document-oriented database that allows for increased flexibility when storing data, allowing for differing structures within collections as opposed to SQL's more rigid framework. This is our preferred method in data storage as it allows for easy handling of large datasets especially with its support of JSON which is essential for our project as it will allow us to utilize arrays for a group of hashtags and the like.

Structure: HTML

- *login.html*
    - Allows the user to log-in to a registered account by sending a POST request to __init__.py which will then pull the associated authentication info from our user database and compares it to that included in the user response.
        - Inaccurate info will prompt user to either reattempt their sign-in or register for an account
        - Accurate info will redirect the user to the landing page
        -
- *register.html*
    - Allows the user to create an account by sending a POST request to __init__.py which will then query our database for any existing users that hold the same username.
        - If matches are detected, user is prompted to enter a new username
        - If no matches are detected, user is redirected to the landing page and the new user information is stored in the database.
- *index.html*
    - Page contains a data limitation dropdown:

        - If General is not selected, a separate dropdown will appear prompting for further limitations which must be completed before user proceeds to specification and metric selections
        - If General is selected, user will directly be prompted to select specifications and metric
        - Specifications and metric dropdowns will contain different content dependent on limitation selected
    - Contains submit button that sends user to data.html
- *data.html*
    - Contains a header and two graphs generated based on previously selected limitations, specifications, and metric
    - Contains a form allowing user to formulate a new inquiry that will generate new graphs based upon entries
        - Will be of the same format as that detailed in index.html
    - Contains all code pertaining to the creation of data visualizations (line graphs, charts, etc.) - uses JavaScript

FEF SELECTION: TAILWIND

- We will make use of Tailwind as our front-end framework. Core elements from Tailwind that we'd like to use include classes and Tailwind's high customizability not offered in either Bootstrap and Foundations.

APIS IN USE

None

DATA VISUALIZATION LIBRARY SELECTION: D3.js

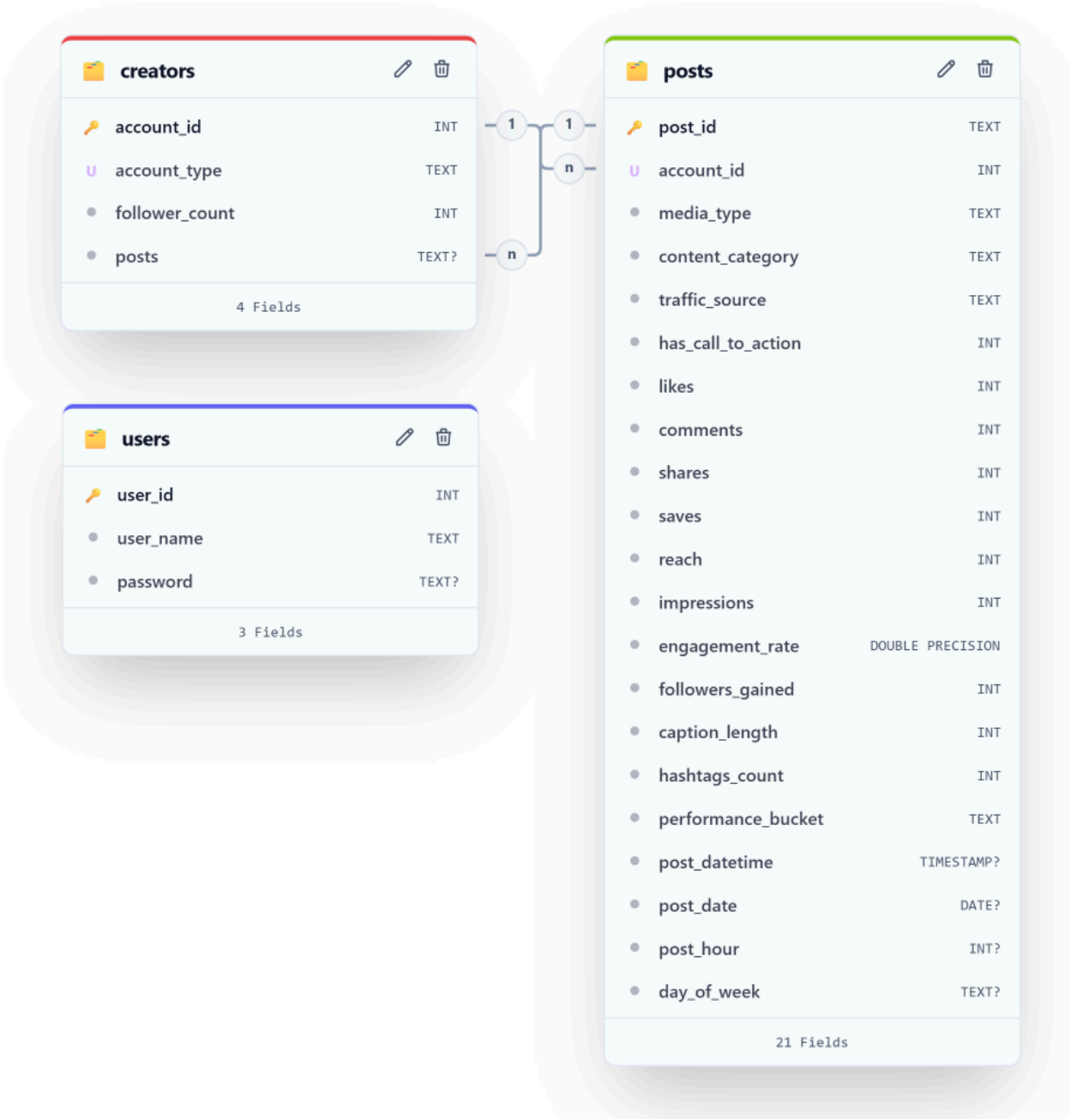
- We will likely be creating visualizations of our data and simulations through the usage of D3.js which provides higher versatility not found with Chart.js. Additionally, its extensive gallery (example code) as well as documentation makes it easy to set something simple up!

TASKS/ASSIGNMENTS:

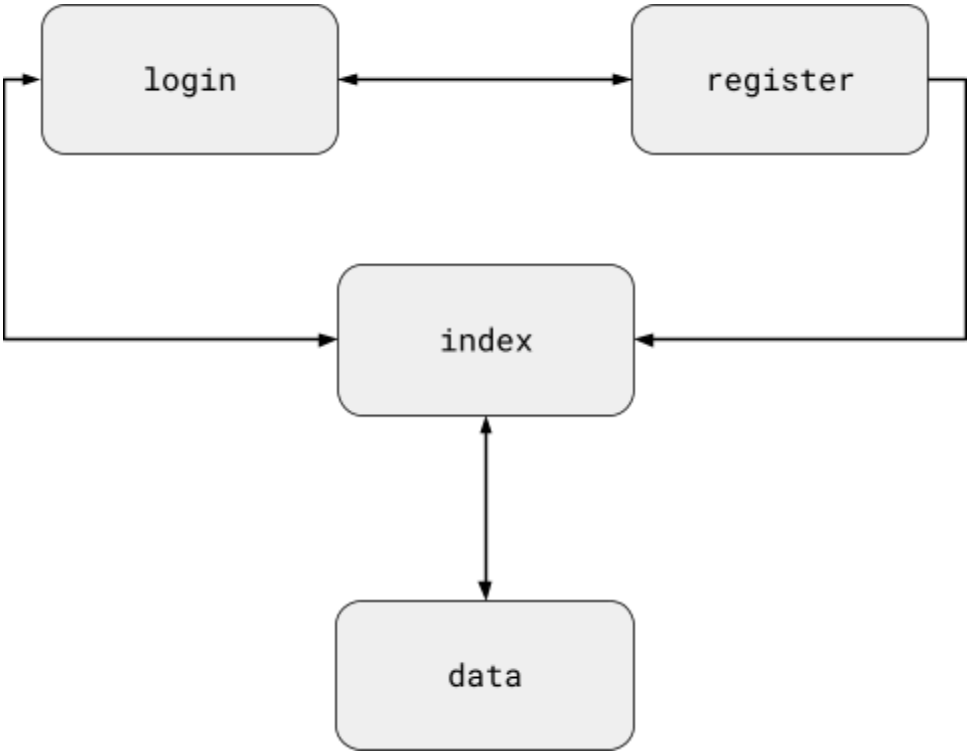
- [ ] Database Setup/Management (Evan)
    - [ ] Pull dataset from Kaggle

- [ ] Set-up MongoDB database/server
- [ ] Backend/Python (Megan)
  - [ ] __init__.py - Main control for website functionality and communication between database and website
  - [ ] data.py - Functions to process data from database into more usable/functional values
- [ ] Server Setup/Management (Joyce, Artemis)
  - [ ] Configure Flask, Nginx, & Gunicorn
  - [ ] Ensure smooth connections between components
- [ ] Frontend/HTML (Sophia)
  - [ ] login.html - Log in user
  - [ ] register.html - Register user
  - [ ] index.html - Selection page to choose data categories to be visualized in data.html
  - [ ] data.html - Data visualization page where all the graphs/plots will be and data storytelling will take place
- [ ] Data Visualization (Evan, Megan)
  - [ ] Solidify choice for data visualization method (Graph, Plot, etc.)
  - [ ] Allow customizability for user (constraints & field types)
- [ ] FEF/Webpage Customization (Joyce, Sophia)
  - [ ] Make website clean and visually appealing
  - [ ] Add functionality through Tailwind buttons and more

DATABASE ORGANIZATION



SITE MAP



COMPONENT MAP